

Lecture 03

Software Re-Engineering



Engr. Maqsood Khatoon

Department of Software Engineering
National University of Computer & Emerging Sciences
maqsood.khatoon@nu.edu.pk

Today's Agenda

- Software Maintenance
 - Reasons of Software Maintenance
 - Types of Software Maintenance (Corrective, Adaptive, Perfective, Preventive Maintenance)
 - Bohem's Software Maintenance Model,

| What is Software Maintenance?

- ✓ Software maintenance is the continuous process of **modifying, updating, and refining** software after its initial launch to ensure it remains functional, secure, and aligned with user needs.
- ✓ It is a critical phase of the Software Development Life Cycle (SDLC), often accounting for 40% to 70% of a system's total lifetime cost.

| Reasons for Software Maintenance

- **Bug Fixes:** Resolving defects and errors discovered by users post-deployment.
- **Security:** Implementing patches to protect against evolving cyber threats and vulnerabilities.
- **Performance:** Optimizing code to improve speed, efficiency, and reliability under load.
- **Environment Adaptation:** Updating software to remain compatible with new operating systems, hardware, or third-party APIs.
- **Evolving Requirements:** Adding new features or modifying existing ones based on user feedback and market trends.

| Types of Software Maintenance

Corrective Maintenance:

A reactive process focused on fixing faults and bugs (e.g., repairing a broken payment gateway or a system crash).

Perfective Maintenance:

Enhancing the software's functionality, usability, or performance based on user demands (e.g., refining the UI/UX or adding new search filters).

Adaptive Maintenance

Modifying the software to keep it usable in a changing environment (e.g., migrating to the Cloud or ensuring compatibility with a new OS version).

Preventive Maintenance

Proactive activities designed to prevent future problems by reducing complexity and improving maintainability (e.g., code refactoring and updating documentation).

| Example

You are part of the maintenance team for a 10-year-old **Student Information System**. The system was recently moved to the cloud, and you must now handle a series of real-time requests.

1. "The system crashes when 500 students try to register simultaneously." → **Corrective** (Fixing bugs).
2. "Migrate the database to support a new encryption standard required by government law." → **Adaptive** (Adapting to external environment).
3. "Students want a 'Dark Mode' and a mobile-friendly dashboard." → **Perfective** (Enhancing functionality).
4. "The code for the login module is messy and prone to future errors; refactor it." → **Preventive** (Proactive optimization).

Classification & Prioritization

Ticket	Classification	Priority	Justification (Boehm's Logic)
Ticket A (Crash)	Corrective	1 (Critical)	High cost of downtime; immediate impact on system reliability.
Ticket B (Encryption)	Adaptive	2 (High)	External legal constraint; system becomes obsolete/illegal if not updated.
Ticket D (Refactor)	Preventive	3 (Medium)	Reduces future maintenance costs by lowering Technical Debt.
Ticket C (Dark Mode)	Perfective	4 (Low)	Aesthetic improvement; lowest ROI compared to system stability.

Software maintenance models

✓ 1. Quick-fix model

A quick-fix model in software maintenance prioritizes rapid bug resolution over a comprehensive solution.

This approach typically involves small, localized changes for immediate issues, typically in emergencies. Tech teams annotate changes with IDs and comments, record them in maintenance history, and merge multiple code points via the assigned change IDs.

| Software maintenance models

✓ 2. Iterative enhancement model

The iterative model is ideal for small-scale app modernization and scheduled maintenance. However, it carries risk as it often overlooks business justification, involving only the software development team. Treating the application target as known, it makes changes based on existing system analysis, best suited for confined targets with minimal cross-impact on other applications or organizations

Software maintenance models

✓ 3. Reuse-oriented model

This approach identifies reusable components in existing systems, extending to the creation of components for broader application use. It encompasses Object and Function reuse, Application System reuse, and Component reuse.

Object and Function reuse involves utilizing software elements and implementing well-defined objects. Application System reuse allows the integration of new components without system alterations. Component reuse involves leveraging pre-existing components, such as code libraries or frameworks, rather than creating new ones in software development.

| Boehm's Software Maintenance Model

Proposed by Barry Boehm in 1983, this model views maintenance as an **economic process** driven by management decisions.

- **Closed-Loop Cycle:** It represents maintenance as a continuous loop where changes are proposed, analyzed, and approved before execution.
- **Cost-Benefit Analysis:** Maintenance tasks are prioritized based on their expected return on investment (ROI) and risk level.
- **Management Centric:** The process is driven by a maintenance manager who must balance objectives (like quality and speed) against environmental constraints and budgets.
- **Effort Estimation:** The model includes formulas (linked to the COCOMO model) to estimate maintenance effort based on **Annual Change Traffic (ACT)**, which is the fraction of software modified each year.